



การหาคู่หรือหาเพื่อนให้น้องแมว

Pawmise

โดย

663380395-4	นางสาวพิชามญช์	พงศ์เศรษฐ์สันต์	กลุ่มการเรียนรู้ที่ 2
663380392-0	นายพงศ์สุพัฒน์	เดิมนันท์	กลุ่มการเรียนรู้ที่ 2
663380413-8	นายอรรณพ	แสงศิลา	กลุ่มการเรียนรู้ที่ 2
663380599-8	นายฉัตรกร	สุวรรณบุตรวิภา	กลุ่มการเรียนรู้ที่ 2

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP363205 ภาษาอนุกรมข้อมูลและการประยุกต์

ภาคเรียนที่ 1 ปีการศึกษา 2568

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

คำนำ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนา ระบบแอปพลิเคชันจับคู่แมว (Cat Tinder Application) ที่ช่วยให้เจ้าของแมวสามารถค้นหาคู่ครองหรือเพื่อนให้แมวของตนได้อย่างสะดวกและปลอดภัย โดยระบบถูกออกแบบให้มีผู้ใช้งานหลัก 2 บทบาท ได้แก่ เจ้าของแมว (Owner) และ ผู้ดูแลระบบ (Admin) เพื่อให้การจัดการข้อมูลแมวและการดูแลระบบเป็นไปอย่างมีประสิทธิภาพและเป็นระบบระเบียบ

ในการดำเนินการพัฒนาแบ่งออกเป็น 2 ส่วนหลัก ได้แก่ ส่วน Backend และ ส่วน Frontend โดยส่วน Backend พัฒนาด้วย Node.js และ Express.js เชื่อมต่อฐานข้อมูล MongoDB ผ่าน Mongoose ORM พร้อมระบบ JWT Authentication เพื่อยืนยันตัวตนและรักษาความปลอดภัยของผู้ใช้งาน รวมถึงมีการใช้ bcryptjs สำหรับการเข้ารหัสผ่าน และกำหนดโครงสร้าง API สำหรับการจัดการข้อมูลแมว โปรไฟล์ผู้ใช้ และระบบจับคู่ ส่วน Frontend พัฒนาด้วย React Native Framework ผ่าน Expo CLI ใช้ TypeScript เพื่อเพิ่มความปลอดภัยของโค้ด และตกแต่งด้วย NativeWind (Tailwind CSS สำหรับ React Native) เพื่อสร้างประสบการณ์ผู้ใช้ (UI/UX) ที่ทันสมัยและตอบสนองรวดเร็ว

จากการพัฒนา ระบบสามารถรองรับการทำงานหลักได้ครบถ้วน ได้แก่ การสมัครสมาชิกและเข้าสู่ระบบ การสร้างและแก้ไขโปรไฟล์แมว การแสดงรายการแมวอื่น ๆ เพื่อทำการ Swipe (Like / Pass) และระบบ จับคู่ (Match) เมื่อมีการกด Like ตรงกันระหว่างสองฝ่าย พร้อมทั้งมีการวางโครงสร้างสำหรับฟีเจอร์ การแชทแบบเรียลไทม์ เพื่อให้ผู้ใช้สามารถติดต่อสื่อสารกันได้หลังจากจับคู่สำเร็จ

แม้ว่าระบบจะสามารถทำงานได้อย่างเสถียรและตอบสนองต่อผู้ใช้ได้อย่างรวดเร็ว แต่ยังมีข้อจำกัดในบางส่วน เช่น ระบบการแจ้งเตือนและระบบแชทที่ยังอยู่ในระหว่างการพัฒนาและระบบจับคู่ตามตำแหน่งที่ตั้ง ในอนาคตควรมีการเพิ่มเติมฟีเจอร์เหล่านี้ รวมถึงการใช้ Machine Learning เพื่อแนะนำคู่แมวที่เหมาะสมตามลักษณะนิสัยหรือสายพันธุ์ ตลอดจนการปรับปรุงระบบความปลอดภัยและการจัดเก็บข้อมูล เพื่อให้แพลตฟอร์มสามารถตอบโจทย์ผู้ใช้และยกระดับเป็น แอปพลิเคชันสำหรับการเชื่อมโยงชุมชนคนรักแมว ได้อย่างสมบูรณ์

คณะผู้จัดทำ

สารบัญ

	หน้า
คำนำ	ก
สารบัญ	๗
สารบัญภาพ	ค
สารบัญตาราง	ง
1. บทนำ	
1.1 ความเป็นมาและที่มาของปัญหาของโปรเจค	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ทฤษฎีและเครื่องมือที่ใช้	1
1.4 โครงสร้างข้อมูล JSON (JSON Map Structure)	2
1.5 โครงสร้าง API (API Map Structure)	6
1.6 ขอบเขตของงานและฟังก์ชันของระบบ	7
2. ภาพรวมการทำงานของระบบ	7
2.1 ฟังก์ชันที่มีในระบบ	7
2.2 โครงสร้าง API และการจัดการข้อมูล	9
2.3 Backend Framework: Express.js (Node.js)	9
2.4 การเชื่อมต่อกับฐานข้อมูล MongoDB	11
บรรณานุกรม	12

สารบัญภาพ

ภาพที่ 1 โครงสร้างแบบ Key-Value

หน้า

1

สารบัญตาราง

	หน้า
ตารางที่ 1 โครงสร้าง API ของ Pawmise	6
ตารางที่ 2 ฟังก์ชันการทำงานโดยแบ่งตามบทบาทผู้ใช้งาน	7

1. บทนำ

1.1 ความเป็นมาและที่มาของปัญหาของโปรเจก

ในปัจจุบัน “แมว” ได้กลายเป็นสัตว์เลี้ยงที่ได้รับความนิยมอย่างแพร่หลาย เจ้าของแมวจำนวนมากไม่เพียงมองแมวเป็นสัตว์เลี้ยงเท่านั้น แต่ยังมองว่าเป็นสมาชิกในครอบครัว ทำให้เกิดความต้องการในการดูแลเอาใจใส่ และหาวิธีสร้างความสุขให้กับแมวของตัวเอง หนึ่งในความต้องการที่พบได้บ่อย คือการหาคู่หรือหาเพื่อนให้แมวเพื่อจุดประสงค์ด้านการผสมพันธุ์หรือการเข้าสังคม อย่างไรก็ตาม การหาคู่แมวที่เหมาะสมในปัจจุบันยังคงทำได้ยาก เจ้าของส่วนใหญ่มักใช้กลุ่มโซเชียลมีเดียหรือฟอรัม ซึ่งมีข้อจำกัดด้านความสะดวก ความเป็นส่วนตัว และการตรวจสอบข้อมูล

ปัญหาดังกล่าว ผู้พัฒนาได้เล็งเห็นโอกาสในการสร้างระบบที่ช่วยอำนวยความสะดวกให้กับเจ้าของแมวในการค้นหาคู่ที่เหมาะสม จึงได้พัฒนาโครงการ “Pawmise” ซึ่งเป็นแอปพลิเคชันบนมือถือที่จำลองหลักการทำงานแบบเดียวกับแอปพลิเคชันจับคู่ (Matching Application) โดยผู้ใช้งานสามารถสร้างโปรไฟล์ของแมว ติดตั้งรูปภาพ ระบุข้อมูลพื้นฐาน และทำการเลื่อน (Swipe) เพื่อค้นหาแมวที่เข้ากันได้ นอกจากนี้ยังมีระบบจับคู่ (Match System) ที่ช่วยให้ผู้ใช้งานที่กด “ถูกใจ” กันทั้งสองฝ่ายสามารถติดต่อกันต่อไปได้

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อพัฒนาแอปพลิเคชันจับคู่แมวที่สามารถให้เจ้าของแมวสามารถหาคู่หรือเพื่อนให้แมวของตนเองได้สะดวกและปลอดภัย
2. เพื่อสร้างระบบที่รองรับการจัดการข้อมูลโปรไฟล์ของแมว เช่น ชื่อ พันธุ์ เพศ อายุ และจุดประสงค์ในการจับคู่
3. เพื่อออกแบบและพัฒนาระบบจับคู่ (Matching System) ที่ทำงานด้วยหลักการ “Like / Pass” แบบเรียลไทม์
4. เพื่อพัฒนา Backend ที่มีระบบยืนยันตัวตน (JWT Authentication) และจัดการข้อมูลด้วยฐานข้อมูล MongoDB อย่างมีประสิทธิภาพ
5. เพื่อวางโครงสร้างสำหรับระบบการสื่อสารแบบเรียลไทม์ (Chat)

1.3 ทฤษฎีและเครื่องมือที่ใช้

1. ภาษานุกรมข้อมูล (JSON – JavaScript Object Notation)
เป็นรูปแบบข้อมูลที่ใช้ในการแลกเปลี่ยนข้อมูลระหว่างระบบ Frontend และ Backend โดยมีโครงสร้างแบบ Key-Value ที่อ่านและเข้าใจง่าย เช่น

ภาพที่ 1 โครงสร้างแบบ Key-Value

```
json
```

```
{ "name": "Mochi", "ageMonths": 12, "gender": "female" }
```

ข้อมูลในระบบทั้งหมดของ Pawmise เช่นโปรไฟล์แมว และข้อมูลของผู้ใช้จะถูกจัดเก็บและแลกเปลี่ยนในรูปแบบ JSON

2. Node.js และ Express.js

ใช้สำหรับพัฒนาเซิร์ฟเวอร์และ RESTful API เชื่อมต่อกับฐานข้อมูล MongoDB รองรับการจัดการข้อมูลผ่าน HTTP Method เช่น GET, POST, PUT, DELETE

3. MongoDB และ Mongoose

เป็นฐานข้อมูลเชิงเอกสาร (Document-based Database) ที่เก็บข้อมูลในรูปแบบ JSON ซึ่งสอดคล้องกับลักษณะการใช้งานของระบบนี้ โดยใช้ Mongoose เป็นตัวช่วยเชื่อมต่อและจัดการ Schema

4. React Native และ Expo

ใช้สำหรับพัฒนาแอปพลิเคชันมือถือแบบ Cross-platform สามารถรันได้ทั้งบน iOS และ Android มีการจัดการ UI ด้วย NativeWind (Tailwind CSS) เพื่อให้การออกแบบสวยงามและตอบสนองรวดเร็ว

5. JWT (JSON Web Token)

ใช้สำหรับการยืนยันตัวตนของผู้ใช้เมื่อเข้าสู่ระบบ โดยฝั่ง Backend จะออก Token ให้ผู้ใช้หลังจากเข้าสู่ระบบสำเร็จ และตรวจสอบ Token ในทุกคำร้องขอที่ต้องการสิทธิ์เข้าถึง

1.4 โครงสร้างข้อมูล JSON (JSON Map Structure)

1.4.1 Cat Model

```
const catSchema = new mongoose.Schema({
  ownerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Owner', required: true, index: true },
  name: { type: String, required: true, trim: true },
  gender: { type: String, enum: ['male', 'female'], required: true, index: true },
  ageYears: { type: Number, min: 0, default: 0 },
  ageMonths: { type: Number, min: 0, max: 11, default: 0 },
  breed: { type: String, required: true },
  color: String,
  traits: [{
    type: String,
    enum: ['playful', 'calm', 'friendly', 'shy', 'affectionate', 'independent', 'vocal', 'quiet']
  }],
  photos: [{
    url: { type: String, required: true },
    publicId: String
  }],
  readyForBreeding: { type: Boolean, default: true },
```

```

vaccinated: { type: Boolean, default: false },
neutered: { type: Boolean, default: false },
notes: String,
location: {
  province: { type: String, default: " " },
  district: String,
  lat: { type: Number, default: 0 },
  lng: { type: Number, default: 0 }
},
active: { type: Boolean, default: true, index: true },
}, { timestamps: true });

```

1.4.2 Matches Model

```

const matchSchema = new mongoose.Schema({
  catAId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Cat',
    required: true,
    index: true },
  ownerAId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Owner',
    required: true,
    index: true },
  catBId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Cat',
    required: true,
    index: true },
  ownerBId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Owner',
    required: true,
    index: true },
  lastMessageAt: {

```



```

    type: Date,
    default: null,
    index: true },
  }, { timestamps: {
    createdAt: true,
    updatedAt: false } }]);

```

1.4.3 Messages Model

```

const messageSchema = new mongoose.Schema({
  matchId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Match',
    required: true,
    index: true },
  senderOwnerId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Owner',
    required: true,
    index: true },
  text: {
    type: String,
    required: true },
  read: {
    type: Boolean,
    default: false,
    index: true }
}, { timestamps: { createdAt: 'sentAt', updatedAt: false } });

```

1.4.4 Owners Model

```

const ownerSchema = new mongoose.Schema({
  // Authentication
  email: { type: String, unique: true, required: true, index: true, lowercase: true, trim: true },
  passwordHash: { type: String, required: true },
  username: { type: String, required: true, unique: true, index: true, trim: true },

```

```
phone: { type: String },
avatarUrl: String,
location: {
  province: { type: String, default: " " },
  district: String,
  lat: { type: Number, default: 0 },
  lng: { type: Number, default: 0 }
},
onboardingCompleted: { type: Boolean, default: false },
active: { type: Boolean, default: true },
}, { timestamps: true });
```

1.4.5 Swipes Model

```
const swipeSchema = new mongoose.Schema({
  swiperOwnerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Owner', required:
true, index: true },
  swiperCatId: { type: mongoose.Schema.Types.ObjectId, ref: 'Cat', required: true,
index: true },
  targetCatId: { type: mongoose.Schema.Types.ObjectId, ref: 'Cat', required: true,
index: true },
  action: { type: String, enum: ['like','pass'], required: true },
}, { timestamps: { createdAt: true, updatedAt: false } });
```

1.5 โครงสร้าง API (API Map Structure)

ตารางที่ 1 โครงสร้าง API ของ Pawmise

Method	Endpoint	รายละเอียด
Register		
POST	/api/auth/register	ลงทะเบียน
POST	/api/auth/login	เข้าสู่ระบบ
GET	/api/auth/me	ดึงข้อมูลผู้ใช้ปัจจุบัน
POST	/api/auth/add-cat	เพิ่มแมว
Owners		
GET	/api/owners/profile	ดึงข้อมูลผู้ใช้
PUT	/api/owners/profile	แก้ไขโปรไฟล์
POST	/api/owners/onboarding	ทำ onboarding
Cat		
GET	/api/cats/feed	ดึงแมวที่สามารถ swipe ได้
GET	/api/cats/my-cats	ดูแมวของฉัน
POST	/api/cats	เพิ่มแมวใหม่
PUT	/api/cats/:id	แก้ไขข้อมูลแมว
DELETE	/api/cats/:id	ลบแมว
Swipes		
POST	/api/swipes	สร้าง swipe (like/pass)
GET	/api/swipes/likes-sent/:catId	ดูว่าแมวของเรากด "ถูกใจ" แมวตัวไหนไปบ้าง
GET	/api/swipes/like-received/:catId	ดูรายการแมวที่กด Like แมวของเรา
Matches		
GET	/api/matches	ดู match ทั้งหมด
DELETE	/api/matches/:id	Unmatch
Messages		
GET	/api/messages/:matchId	ดูข้อความ
POST	/api/messages	ส่งข้อความ
PUT	/api/messages/:matchId/read	ทำเครื่องหมายว่าอ่านแล้ว

1.6 ขอบเขตของงานและฟังก์ชันของระบบ

1.6.1 ขอบเขตของระบบ

1) ส่วนของผู้ใช้งาน (Frontend):

พัฒนาโดยใช้ React Native Framework ผ่าน Expo CLI เพื่อให้สามารถใช้งานได้ทั้งระบบ iOS และ Android มีหน้าจอสำหรับการลงทะเบียนเข้าสู่ระบบ การสร้างโปรไฟล์แมว การดูข้อมูลแมวอื่น ๆ และการเลื่อน (Swipe) เพื่อจับคู่

2) ส่วนของระบบหลังบ้าน (Backend):

พัฒนาโดยใช้ Node.js และ Express.js เชื่อมต่อกับฐานข้อมูล MongoDB ผ่าน Mongoose ORM มีระบบจัดการข้อมูลผู้ใช้และข้อมูลแมว พร้อมระบบยืนยันตัวตนด้วย JWT Token และเข้ารหัสผ่านด้วย bcryptjs

ขอบเขตการทำงานในเวอร์ชันปัจจุบัน ครอบคลุมถึงฟีเจอร์การสมัครสมาชิก การเข้าสู่ระบบ การจัดการโปรไฟล์ การจับคู่แมว และการเตรียมโครงสร้างสำหรับฟีเจอร์แชทและระบบระบุตำแหน่งที่ยังอยู่ในระหว่างการพัฒนา

1.6.2 ฟังก์ชันการทำงาน (แบ่งตามบทบาทผู้ใช้งาน)

ตารางที่ 2 ฟังก์ชันการทำงานโดยแบ่งตามบทบาทผู้ใช้งาน

ผู้ใช้งาน	ฟังก์ชันหลัก
Owner (เจ้าของแมว)	- สมัครสมาชิก / เข้าสู่ระบบ - สร้างและแก้ไขโปรไฟล์แมว - ดูข้อมูลแมวอื่น ๆ ที่อยู่ในระบบ - กด Like / Pass เพื่อจับคู่ - ดูผลการ Match - ระบบ Chat

2. ภาพรวมการทำงานของระบบ

Pawmise เป็นแอปพลิเคชันมือถือที่พัฒนาด้วย React Native และ Expo ซึ่งมีแนวคิดคล้ายกับแอปหาคู่สำหรับคน แต่ออกแบบมาเพื่อช่วยเจ้าของแมวในการหาคู่ให้กับสัตว์เลี้ยงของตนเอง ระบบนี้ใช้สถาปัตยกรรมแบบ Client-Server โดย Backend พัฒนาด้วย Node.js, Express และ MongoDB พร้อมระบบจัดการรูปภาพผ่าน Cloudinary

2.1 ฟังก์ชันที่มีในระบบ

1) การเริ่มต้นใช้งานและระบบยืนยันตัวตน

เมื่อผู้ใช้เปิดแอปพลิเคชันครั้งแรก ระบบจะนำไปสู่หน้าลงทะเบียน ซึ่งต้องกรอกข้อมูลพื้นฐานอย่างชื่อผู้ใช้ อีเมล รหัสผ่าน เบอร์โทรศัพท์ ระบบจะตรวจสอบความถูกต้องของข้อมูล โดยเฉพาะชื่อผู้ใช้ที่ต้องมีความยาวอย่างน้อยสามตัวอักษรและประกอบด้วยเฉพาะตัวอักษร ตัวเลข และเครื่องหมายขีดกลางเท่านั้น เมื่อข้อมูลถูกต้อง Backend จะเข้ารหัสผ่านและสร้างบัญชีผู้ใช้ใหม่ในฐานข้อมูล จากนั้นสร้าง JWT Token เพื่อใช้

ในการยืนยันตัวตนและส่งกลับมายังแอป Token นี้จะถูกเก็บไว้ใน AsyncStorage ของเครื่องเพื่อใช้ในการเข้าสู่ระบบอัตโนมัติในครั้งถัดไป

สำหรับผู้ใช้งานที่มีบัญชีอยู่แล้ว สามารถเข้าสู่ระบบด้วยอีเมลและรหัสผ่าน ระบบจะตรวจสอบข้อมูลและสร้าง Token ใหม่ทุกครั้งที่เข้าสู่ระบบ ทุกครั้งที่เปิดแอป ระบบจะตรวจสอบ Token ที่เก็บไว้ว่ายังใช้งานได้หรือไม่ โดยเรียก API เพื่อยืนยันสถานะของผู้ใช้ หาก Token หมดอายุหรือผู้ใช้ถูกลบออกจากระบบ แอปจะล้างข้อมูลและนำกลับไปยังหน้าเข้าสู่ระบบ

2) การจัดการโปรไฟล์ผู้ใช้

หลังจากลงทะเบียนสำเร็จ ผู้ใช้จะมีโปรไฟล์ส่วนตัวที่ประกอบด้วยข้อมูลต่างๆ เช่น ชื่อ ผู้ใช้ อีเมล เบอร์โทรศัพท์ รูปโปรไฟล์ ผู้ใช้สามารถแก้ไขข้อมูลส่วนตัวได้โดยกดปุ่มแก้ไขในหน้าโปรไฟล์ ระบบจะเปลี่ยนเป็นโหมดแก้ไขที่สามารถปรับเปลี่ยนชื่อผู้ใช้ เบอร์โทรศัพท์ ก่อนบันทึกข้อมูล ระบบจะตรวจสอบว่าชื่อผู้ใช้ใหม่ไม่ซ้ำกับผู้อื่น

การอัปโหลดรูปโปรไฟล์ทำได้โดยเลือกรูปจากเครื่อง ระบบจะส่งรูปไปยัง Cloudinary ซึ่งเป็นบริการจัดเก็บรูปภาพบนคลาวด์ เมื่อได้ URL กลับมาจะบันทึกลงในฐานข้อมูลและแสดงในหน้าโปรไฟล์ทันที การใช้ Cloudinary ช่วยลดภาระของเซิร์ฟเวอร์และทำให้การโหลดรูปภาพรวดเร็วขึ้น

3) การเพิ่มและจัดการข้อมูลแมว

หัวใจสำคัญของแอปพลิเคชันคือการเพิ่มข้อมูลแมว ซึ่งออกแบบเป็นกระบวนการสี่ขั้นตอนเพื่อให้ง่ายต่อการใช้งานและไม่ทำให้ผู้ใช้รู้สึกท้อถอย ขั้นตอนแรกคือการเลือกรูปภาพของแมว ผู้ใช้สามารถเลือกได้หนึ่งถึงห้ารูปจากคลังรูปในเครื่อง รูปที่เลือกจะแสดงเป็นพรีวิวและสามารถลบออกได้หากต้องการเปลี่ยนแปลง ระบบกำหนดให้ต้องมีรูปอย่างน้อยหนึ่งรูปก่อนไปขั้นตอนถัดไป

ขั้นตอนที่สองเป็นการกรอกข้อมูลพื้นฐานของแมว ได้แก่ ชื่อ เพศ และอายุ โดยอายุสามารถระบุทั้งปีและเดือนได้แยกกัน ข้อมูลเหล่านี้เป็นข้อมูลบังคับที่จำเป็นสำหรับการแสดงผลในระบบ ขั้นตอนที่สามเน้นไปที่สายพันธุ์และสุขภาพ ผู้ใช้เลือกสายพันธุ์จากรายการที่มีให้ หากไม่พบสายพันธุ์ที่ต้องการสามารถเลือก "อื่นๆ" และระบุชื่อสายพันธุ์เอง นอกจากนี้ยังสามารถระบุสีขนและสถานการณั้วัดชินของแมวได้

ขั้นตอนสุดท้ายคือการเลือกนิสัยของแมวและเพิ่มหมายเหตุ ผู้ใช้สามารถเลือกนิสัยได้หลายอย่าง เช่น ขี้เล่น เป็นมิตร สงบ เป็นต้น หมายเหตุเป็นช่องเสริมที่ผู้ใช้สามารถบอกรายละเอียดเพิ่มเติมเกี่ยวกับแมว เมื่อกรอกข้อมูลครบทั้งสี่ขั้นตอน ระบบจะรวบรวมข้อมูลทั้งหมดเป็น FormData แปลงรูปภาพเป็น Blob และส่งไปยัง Backend Backend จะอัปโหลดรูปภาพไป Cloudinary และบันทึกข้อมูลแมวพร้อม URL ของรูปลงในฐานข้อมูล MongoDB

ข้อมูลแมวในระบบประกอบด้วยรหัสเฉพาะ ชื่อเจ้าของ (เป็นชื่อผู้ใช้แบบข้อความ) ชื่อแมว เพศ อายุ สายพันธุ์ สี นิสัย รูปภาพ สถานะวัดชิน สถานะทำหมัน หมายเหตุ และตำแหน่งที่ตั้ง นอกจากนี้ยังมีฟิลด์ระยะทางที่คำนวณจากตำแหน่งของผู้ใช้ปัจจุบัน ซึ่งใช้ในการกรองและจัดเรียงแมวที่แสดงในฟีด

4) ระบบการหาคู่และการ Swipe

เมื่อผู้ใช้เพิ่มข้อมูลแมวเสร็จแล้ว จะสามารถเข้าสู่หน้าหลักของแอปที่แสดงฟีดแมว ระบบจะเรียก API เพื่อดึงข้อมูลแมวใกล้เคียงมาประมาณยี่สิบตัว Backend จะกรองแมวโดยอัตโนมัติเพื่อไม่แสดงแมวของผู้ใช้เอง แมวที่เคยปิดไปแล้ว หรือแมวที่ยูนอร์กสมิทที่กำหนด แมวที่แสดงจะต้องมีสถานะเป็น Active เท่านั้น

หน้าจอแสดงแมวในรูปแบบการ์ดซ้อนกัน โดยแสดงสามใบพร้อมกัน การ์ดหน้าสุดสามารถปิดได้ ส่วนสองการ์ดข้างหลังแสดงเป็นพริ้วเล็กน้อยเพื่อให้ผู้ใช้รู้ว่าแมวตัวถัดไปรออยู่ เมื่อผู้ใช้ปิดการ์ดไปทางซ้าย หมายถึงการ "ผ่าน" ไม่สนใจ ระบบจะเรียก API เพื่อบันทึกการปิดนี้ลงฐานข้อมูล ถ้าปิดไปทางขวา หมายถึง "ถูกใจ" ระบบจะบันทึกการถูกใจและตรวจสอบว่าเจ้าของแมวอีกฝ่ายได้ถูกใจแมวของเรากลับหรือไม่

หากทั้งสองฝ่ายถูกใจกัน ระบบจะสร้างการ Match โดยอัตโนมัติ ข้อมูล Match ประกอบด้วยข้อมูลของแมวทั้งสองตัวและเจ้าของทั้งสองฝ่าย พร้อมเวลาที่สร้าง Match และเวลาข้อความล่าสุดเมื่อเกิด Match ระบบจะแสดงหน้าต่างป๊อปอัพแสดงความยินดี พร้อมรูปแมวทั้งคู่ มีปุ่มให้เลือกที่จะ "ส่งข้อความ" ทันทีหรือ "ภายหลัง" หากเลือกส่งข้อความจะถูกนำไปยังหน้าแชท

5) ระบบข้อความและการสื่อสาร

มีรายการ Match ทั้งหมดให้ผู้ใช้เลือกคุยกับคู่ที่ต้องการ มีห้องแชทสำหรับแต่ละคู่ Match การทำงานแบบ Real-time ผ่าน Socket.IO เพื่อให้ข้อความถึงกันทันที มีการติดตามสถานะว่าข้อความถูกอ่านแล้วหรือยัง และมี Push Notification เพื่อแจ้งเตือนเมื่อมีข้อความใหม่

2.2 โครงสร้าง API และการจัดการข้อมูล

2.2.1 การเพิ่มข้อมูล (Create)

- 1) การเพิ่มข้อมูล (Create) (API: POST /cats)
- 2) สร้าง Match ใหม่ (API: POST /swipes (ตรวจสอบและสร้าง Match อัตโนมัติ))
- 3) ส่งข้อความใหม่ (API: POST /messages)

2.2.2 การอ่านข้อมูล (Read)

- 1) ดูฟีดแมว (API: GET /cats/feed)
- 2) ดูรายการแมวของตัวเอง (API: GET /cats/my-cats)
- 3) ดูรายการ Match (API: GET /matches)
- 4) ดูประวัติการสนทนา (API: GET /messages/:matchId)

2.2.3 การแก้ไขข้อมูล (Update)

- 1) แก้ไขข้อมูลโปรไฟล์ (API: PUT /owners/profile)
- 2) แก้ไขข้อมูลแมว (API: PUT /cats/:id)

2.2.4 การลบข้อมูล (Delete)

- 1) ลบแมว (API: DELETE /cats/:id)

2.3 Backend Framework: Express.js (Node.js)

ระบบ Pawmise พัฒนาด้วย Express Framework ของ Node.js ซึ่งเป็น Framework ที่มีความยืดหยุ่นและเรียบง่าย เหมาะกับการสร้าง RESTful API สำหรับเชื่อมต่อกับฐานข้อมูล MongoDB ซึ่งช่วยให้สามารถกำหนด Routing, Middleware, Authentication และการจัดการคำขอ (Request/Response) ได้ด้วยตนเอง ทำให้เข้าใจโครงสร้างการสื่อสารของระบบเว็บแอปพลิเคชันอย่างชัดเจน

```
const express = require('express');
const http = require('http');
const cors = require('cors');
require('dotenv').config();

// Import connectDB
const connectDB = require('./config/db');
const { initializeSocket } = require('./socket/socketServer');

const app = express();
const server = http.createServer(app);

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
app.use('/api/auth', require('./routes/authRoute'));
app.use('/api/owners', require('./routes/ownersRoute'));
app.use('/api/cats', require('./routes/catsRoute'));
app.use('/api/swipes', require('./routes/swipesRoute'));
app.use('/api/matches', require('./routes/matchesRoute'));
app.use('/api/messages', require('./routes/messagesRoute'));

// เชื่อมต่อ MongoDB ก่อนเริ่ม server
connectDB();

// Initialize Socket.IO
const io = initializeSocket(server);

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', message: 'Server is running' });
});
```

```
// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    message: 'Something went wrong!',
    error: process.env.NODE_ENV === 'development' ? err.message : undefined
  });
});

const PORT = process.env.PORT || 5000;

server.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
  console.log(`Socket.IO server is ready`);
  console.log(`API URL: http://localhost:${PORT}`);
});
```

2.4 การเชื่อมต่อกับฐานข้อมูล MongoDB

ระบบ Backend เชื่อมต่อกับฐานข้อมูล MongoDB Compass โดยกำหนดค่าการเชื่อมต่อในไฟล์ .env

```
MONGODB_URI=mongodb+srv://pichamonph_db_user:5dau1tQs2eYw8FWF@cat-tinder-cluster.5jekyot.mongodb.net/?retryWrites=true&w=majority&appName=cat-tinder-cluster
```

ข้อมูลที่บันทึกใน MongoDB จะถูกจัดเก็บในรูปแบบ JSON Document เช่น Collection cats, owners, matches

บรรณานุกรม

Expo. (n.d.). *Expo Documentation*. <https://expo.dev/>

Express. (n.d.). *Express - Node.js web application framework*. <https://expressjs.com/>

Meta Platforms, Inc. (n.d.). *React Native · Learn once, write anywhere*. <https://reactnative.dev/>

NativeWind. (n.d.). *NativeWind - Tailwind CSS for React Native*. <https://www.nativewind.dev/>

Node.js Foundation. (n.d.). *Node.js*. <https://nodejs.org/en>